

ProgressSync AI

Master Project & Development Document — Final Consolidated Version

SIH26122 · Intelligent Data Capture & Schedule-Linking Layer for Infrastructure Project Management · Oil India Limited · Smart Automation · Software
Version 1.0 · 16 September 2026 · Supersedes: Project Brief, PS Alignment & Gap Analysis, Additional Features & Feasibility Analysis

POSITIONING

ProgressSync AI is an explainable execution-intelligence bridge that converts heterogeneous site reports into validated L5/L6 schedule updates, recomputes the critical path from real actuals, preserves uncertainty instead of hiding it, and builds reusable project memory.

The clause that is new in this version — and the one that most separates this submission from the rest of the SIH26122 field — is **"recomputes the critical path from real actuals."** See Section 2.

0. How to use this document

This document consolidates and replaces the team's three working files. Where they disagreed, this version is authoritative. It is written to serve three audiences at once:

- **The team, as a build spec.** Sections 4–11 are implementable as written: schemas, scoring formula, thresholds, API contract, folder layout.
- **The evaluator, as a defence.** Sections 1, 2, 14 and 15 answer "why you and not the other teams" and the questions a problem-setting organisation will actually ask.
- **The team lead, as a schedule.** Section 12 is a dated day-by-day plan to the 25 September deadline with named owners and a cut order.

Three things changed materially from the earlier drafts, and they are the reason this document exists:

1. **A new architectural layer was added** — schedule graph + CPM recompute (Section 4, Layer 6). The earlier brief stopped at "update the actual date." That answers only half the PS.
2. **The five PS-language gaps were folded into the spec itself** rather than sitting in a separate remediation list — voice, discipline as a first-class field, recurring delay causes, near-real-time latency, and the fourth input type.
3. **The feasibility list was converted into build tasks with owners and dates**, not a menu of optional improvements.

1. The problem statement, and what it actually demands

1.1 Official framing

Field	Value
Problem Statement ID	SIH26122
Title	Intelligent Data Capture & Schedule-Linking Layer for Infrastructure Project Management: Real-Time Actual Progress Tracking (Planning-to-Execution Bridge)
Organisation	Oil India Limited (OIL)
Category / Theme	Software / Smart Automation
Disciplines named in the PS	Civil; piping; static equipment; rotating equipment; electrical; instrumentation; HSE
Prototype allowance	Synthetic/sample data permitted; 2–3 input formats sufficient; production-grade OCR/ASR explicitly not required

1.2 The four failure modes the PS names

Failure mode	What it means on site	Which layer answers it
Information silos & lag	DPRs, contractor spreadsheets and site notes take days or weeks to become a schedule update.	Layers 1–3 + latency instrumentation (\$4, \$13)
Granular activity disconnect	The plan is L5/L6 with formal IDs; the field describes the same work in its own vocabulary and at finer granularity.	Layer 4 — the matching engine (\$6)
Manual overhead & unverifiable reporting	Planners decipher ambiguous updates by hand; nothing is auditable afterwards.	Layer 5 review + audit trail (\$4, \$9)
No early warning	Delays compound invisibly until a milestone is already breached.	Layer 6 — CPM recompute (\$4, \$7)

CRITICAL READ

The fourth failure mode is the one most teams will miss. Matching a field report to an activity ID and writing an actual date *records history*. It does not produce an **early warning**. An early warning requires knowing whether the slipped activity was on the critical path and which downstream milestone moves as a result. That is a schedule-graph and CPM problem, not an NLP problem — and it is cheap to build once the activity catalogue exists.

1.3 The canonical hard case

Baseline schedule row	Field report text
PIP-24-EL-047 – Erect Line 24-XX-102 from Rack R03 to Vessel V12	"Spool XX102 erected between rack 3 and V12 by piping crew. Completed at 16:45."

Correct system behaviour: link the two, attach a confidence score, and display the evidence that produced it — not a black-box yes/no. Incorrect behaviour, and the single most damaging failure in this domain, is a **confident wrong link**: a silent update that corrupts the progress record and the forecast built on it. Every threshold decision in Section 6 is tuned against that risk, not against raw accuracy.

2. Competitive landscape and the differentiation decision

SIH26122 is a visibly popular problem statement. Several teams have public repositories, and their scope is now knowable. Building without reading it is how a strong team gets screened out for being indistinguishable.

2.1 What the field is building

Observed approach	Typical scope	Implication for us
Ingestion → NLP extraction → embedding + fuzzy match → confidence → planner review → dashboard	The consensus architecture. Present in essentially every serious public repo, including ours as originally briefed.	This is table stakes, not a differentiator . Doing it well earns no separation on its own.
React/Next + FastAPI + Postgres/SQLite + sentence-transformers + RapidFuzz	The consensus stack, near-universally.	Stack choice cannot differentiate. Do not spend a day arguing about it.
Voice input via the browser Web Speech API	Already anticipated by competing teams as the cheap way to satisfy the PS "or voice" clause.	Necessary to avoid a gap, but expect no credit for novelty.
Schedule DAG + Critical Path Method recompute, critical-path diff, downstream at-risk flagging	Present in at least one advanced competing implementation. Absent from most.	This is the real contested ground. We must have it.
Measured benchmark with a held-out split	Rare. Most submissions state an accuracy number with no test protocol behind it.	Cheap for us, and it is the difference between a claim and evidence.

2.2 Our three defensible claims

Everything below is either already in our brief or reachable within the remaining build window. Nothing here requires a capability we do not have.

1. **Decision quality under ambiguity, not match accuracy.** We auto-accept on *margin* — the separation between the top candidate and the runner-up — not on a single threshold. A near-tie between two similar activities is precisely the "same terminology, two activities" failure case, and a margin rule detects it structurally. We can answer "what is your false-positive rate on auto-links?" with a number from a held-out set. Most teams cannot answer it at all.
2. **Closed loop to forecast.** We do not stop at writing an actual date. We re-run CPM on the updated graph, diff the critical path against the previous snapshot, and surface which downstream activities and milestones moved. This converts the deliverable from a *record-keeping tool* into an *early-warning system* — which is what the PS's fourth failure mode asks for.
3. **Explainability as an artifact, not an adjective.** Four sub-scores are displayed individually; the terminology map is a visible, versioned file the matcher demonstrably consults; every update carries old value, new value, source document span, user and timestamp. A reviewer can inspect the reasoning rather than take it on trust.

2.3 What we explicitly refuse to do

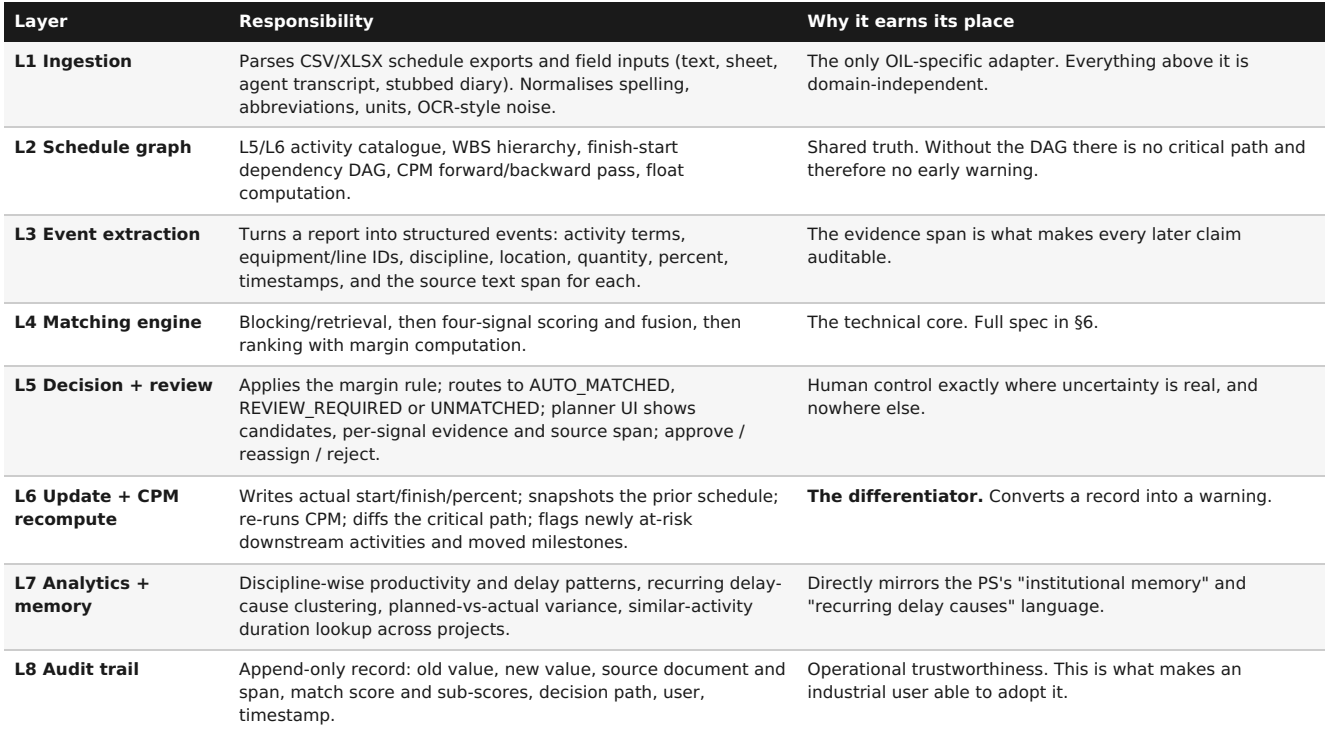
- No production-grade OCR or ASR. The PS waives both; building them burns days and earns nothing.
- No trained fusion model. There is no labelled dataset to train on before the deadline, and a hand-justified weighting is more defensible than a model fitted to synthetic data we generated ourselves.
- No microservices, Kubernetes, or auth system before the matcher and the recompute engine work end to end.
- No generic chatbot. Any conversational surface must write into the same event → match → update pipeline as document ingestion, or it is demo theatre.
- No broadening into a general project-management platform. The PS is narrow on purpose; that narrowness is our advantage.
- No accuracy claim that did not come out of a run in our own repository.

3. Scope lock for v1.0

In scope — must exist by 24 September	Out of scope for v1.0 — stated as future work
Primavera/MS Project CSV/XLSX export ingestion → L5/L6 activity catalogue with dependencies	Live two-way Primavera P6 / PAIMANA API integration
Four input surfaces: free text, spreadsheet, conversational/voice Time Agent, stubbed scanned diary	Real OCR pipeline (Tesseract/PaddleOCR) and real ASR
Deterministic event extraction: IDs, discipline, location, quantity, progress, date/time, evidence span	Fine-tuned domain NER model
Multi-signal confidence matching with margin-based auto-accept	Learned fusion / learning-to-rank
Planner review queue with per-signal evidence breakdown	Role-based access control, SSO, multi-tenant isolation
Schedule update + immutable audit trail	Digital signature / blockchain-style ledger
CPM recompute, critical-path diff, downstream at-risk flagging	Monte Carlo schedule risk simulation
Discipline-wise analytics, recurring delay-cause detection, institutional memory queries	Predictive ML delay forecasting trained on historical projects
Measured matching benchmark on a held-out split	Field pilot validation with real OIL data
Native mobile app for supervisors	(Out — the Time Agent is responsive web)

Discipline rule: anything not in the left column does not get built before 24 September, regardless of how good the idea is. Add it to the right column instead and present it as future work — a well-argued future-work slide scores under the SIH criteria; a half-finished feature does not.

Eight layers. Layer 2 (the schedule graph) is shared state referenced by Layers 4 and 6, not a sequential stage. The domain-independent core is Layers 3-7; Layer 1 is the only OIL-specific adapter, and drawing it that way is what answers the "scale of impact" criterion.



5. Data model

Freeze this on day one. Every downstream task depends on it, and schema churn mid-week is the most common way a hackathon build stalls. Note that `discipline` is a first-class column on three tables, not a derived tag — this is a direct PS requirement.

```
projects(id, name, client, start_date, target_finish)

activities(
    id, project_id,          -- the L5/L6 catalogue
    activity_code,          -- e.g. PIP-24-EL-047
    description,
    wbs_path,               -- L1>L2>L3>L4>L5>L6
    level,                  -- 5 or 6
    discipline,             -- ENUM: civil | piping | static Equip |
                            --       rotating Equip | electrical |
                            --       instrumentation | hse
    location_tag,           -- rack/unit/area identifier
    planned_start, planned_finish, planned_duration_days,
    actual_start, actual_finish, percent_complete,
    status,                 -- not_started | in_progress | complete
    embedding               -- cached vector of description
)

activity_dependencies(predecessor_id, successor_id, type, lag_days)

cpm_state(
    activity_id, snapshot_id, -- recomputed, never hand-edited
    early_start, early_finish, late_start, late_finish,
    total_float, is_critical
)

schedule_snapshots(id, project_id, created_at, trigger_event_id,
    project_finish_date, critical_path_json)

field_reports(id, project_id, source_type, -- text|spreadsheet|agent|diary_stub
    raw_content, submitted_by, submitted_at, file_ref)

execution_events(
    id, report_id,          -- one report yields 1..n events
    raw_text_span, normalised_text,
    extracted_ids_json, discipline, location_tag,
    quantity, unit, percent_reported,
    event_timestamp, extraction_confidence
)

match_candidates(
    id, event_id, activity_id, rank,
    score_id, score_lexical, score_semantic, score_context,
    temporal_factor, fused_score, margin_to_next,
    evidence_json           -- which tokens/terms drove each sub-score
)

match_decisions(
    id, event_id, chosen_activity_id, -- nullable for UNMATCHED
    decision,                        -- AUTO_MATCHED | REVIEW_REQUIRED |
                                    -- UNMATCHED | PLANNER_CONFIRMED |
                                    -- PLANNER_REASSIGNED | PLANNER_REJECTED
    fused_score, margin, decided_by, decided_at
)

terminology_map(
    id, project_id, field_term, canonical_term, discipline, version, source)

delay_causes(id, event_id, activity_id, cause_label, cause_text_span,
    detected_at, discipline)

audit_log(id, entity_type, entity_id, field_changed,
    old_value, new_value, source_report_id, source_span,
    match_score, actor, actor_type, created_at)
```

Enum discipline, do not free-text it. The PS names exactly seven disciplines. A constrained enum makes discipline usable as a hard blocking constraint in Section 6, makes discipline-wise analytics trivially correct, and makes the dashboard's risk heatmap meaningful. A free-text column would quietly destroy all three.

6. The matching engine — full specification

6.1 Pipeline

1. **Normalise.** Case, whitespace, unit forms (24" → 24-inch), digit/letter confusions from OCR-style noise, abbreviation expansion via the terminology map.
2. **Extract.** Activity codes, equipment/line/spool IDs, location tokens, discipline signals, dates and times, quantities and percentages — each with the source character span retained.

3. **Block (retrieve, don't scan).** Filter to the project; apply discipline as a hard constraint where confidently extracted; then a k-NN cosine lookup over pre-computed activity description embeddings to take the **top ~20 candidates**. Full scoring runs on those twenty only.
4. **Score four signals** on each candidate (§6.2).
5. **Fuse** with fixed, justified weights (§6.3).
6. **Rank, compute margin** between rank 1 and rank 2.
7. **Decide** by the margin rule (§6.4).
8. **Persist all candidates and all sub-scores**, not just the winner. This is what the evidence panel renders and what the benchmark analyses.

Why blocking matters even at demo scale: brute-force cosine over a few hundred activities is instantaneous, so blocking is not a performance necessity today. It is a *credibility* necessity — "retrieve-then-rank" is the standard entity-resolution pattern, and it gives an honest answer to "does this scale to a 40,000-activity schedule?" It costs perhaps two hours to implement.

6.2 The four signals

Signal	Computation	What it catches / misses
ID match score_id	Exact and near-exact token match on activity codes, line numbers, spool IDs, equipment tags. Normalised comparison ($XX102 \approx 24-XX-102$). Binary-ish: 1.0 exact, 0.7 partial/substring, 0 absent.	Nearly unambiguous when present. Frequently absent — supervisors rarely quote activity codes.
Lexical score_lexical	Token-set ratio and partial ratio via RapidFuzz over normalised text, after terminology-map substitution.	Catches typos and reordering. Fails completely on true synonyms ("spool erected" vs "erect line").
Semantic score_semantic	Cosine similarity between the sentence embedding of the normalised field text and the cached activity description embedding. Local MiniLM-class model via sentence-transformers.	Catches the synonym case, which is the PS's own example. Can be over-confident between two genuinely similar activities — which is why the margin rule exists.
Context score_context	Weighted agreement on discipline, location tag, parent WBS node, and sequence plausibility (are the activity's predecessors complete?).	Disambiguates near-duplicate descriptions in different areas — the highest-value signal on realistic adversarial data.
Temporal temporal_factor	Multiplier, not an additive term. 1.0 if the event timestamp falls within the planned window $\pm$ tolerance; decays to a floor of 0.6 as the distance grows.	Penalises linking today's report to an activity planned six months out, without hard-blocking a genuine early or late finish.

6.3 Fusion — fixed weights, deliberately not learned

```
base = 0.30 * score_id
      + 0.15 * score_lexical
      + 0.25 * score_semantic
      + 0.30 * score_context

fused = base * temporal_factor

if discipline_extracted and discipline_mismatch:
    fused = fused * 0.50      # strong penalty, not a hard block
```

The justification to give an evaluator, in one sentence: the weights reflect how reliable each signal is in this domain — an explicit ID match is close to unambiguous, structural context (discipline, location, WBS parent) is highly reliable because it comes from the plan rather than from prose, semantic similarity is informative but the most prone to confident error, and free-text lexical overlap is the weakest evidence of all. We do not train these weights because there is no labelled corpus of real OIL field reports to train them on, and fitting them to synthetic data we generated ourselves would produce a number that means nothing. They are documented as design choices, to be calibrated against real data in a pilot.

6.4 The decision rule — margin, not threshold

```
top    = candidates[0].fused
second = candidates[1].fused if len(candidates) > 1 else 0.0
margin = top - second

if top >= 0.82 and margin >= 0.12:  -> AUTO_MATCHED
elif top >= 0.45:                  -> REVIEW_REQUIRED
else:                              -> UNMATCHED    (preserved, never dropped)
```

This is the single most important design decision in the project, and the one to lead with when questioned.

A single-threshold system auto-accepts whenever the best score is high. But the most dangerous case in this domain is two *similar* activities — the same line erected at two different racks — where the top score is high *and* the runner-up is almost as high. A threshold rule accepts confidently and wrongly. A margin rule sees the near-tie, recognises genuine ambiguity, and routes to a human.

The one-line version: **"we auto-match on separation, not on score, because a close second place is the definition of ambiguity."** It costs one subtraction in code that was already ranking candidates.

Both numbers (0.82, 0.12) are starting points to be calibrated on the held-out benchmark in Section 11, then frozen. Report them as calibrated design parameters, never as tuned accuracy results.

6.5 Terminology map as a visible artifact

Ship the abbreviation/synonym map as an inspectable, versioned file (`data/terminology_map.v1.yaml`) surfaced in the UI, not a hidden dictionary inside the matcher. During the demo, show the entry that fired — `spool → piping segment` — next to the match it produced. This converts "we handle domain jargon" from an assertion into something the evaluator can read. It costs nothing: the map is being built for Phase 0 regardless; this is only exposure.

7. CPM recompute and early warning

This layer did not exist in the original brief. It is the highest-value addition in this document, it closes the PS's fourth failure mode, and it is roughly a day of work because the dependency data already arrives inside the Primavera export.

7.1 What it does

1. Build a directed acyclic graph of L5/L6 activities from `activity_dependencies` (networkx; mature, exact, not a black box).
2. Forward pass → early start/finish. Backward pass → late start/finish. Total float = LS – ES. Critical set = activities with zero float.
3. On every confirmed actual update: snapshot the current CPM state, substitute actual durations and dates for the affected activity, re-run both passes.
4. Diff against the snapshot and emit: **(a)** new projected project finish date and the delta in days; **(b)** activities that entered or left the critical path; **(c)** downstream activities whose float dropped below a warning band; **(d)** milestones whose dates moved.
5. Write the diff to the audit log and push it to the dashboard.

7.2 Why this wins the room

Without this layer	With this layer
"We linked the report to activity PIP-24-EL-047 and marked it 100% complete."	"We linked the report to PIP-24-EL-047, which finished two days late. That activity had zero float, so the critical path has shifted through the instrumentation chain, three downstream activities are now at risk, and mechanical completion has moved from 14 March to 16 March."
A record of what happened.	A warning about what is going to happen.
Answers PS failure modes 1–3.	Answers all four, including the one the PS calls "lack of early warning."

Honesty guard: CPM is deterministic arithmetic on a graph, not AI, and it should be described that way. Claiming it as machine learning is exactly the kind of overreach that a domain evaluator from Oil India Limited — people who run Primavera daily — will catch instantly. Present it as: correct, standard project-controls mathematics, applied automatically the moment a field report lands instead of at the next weekly review.

8. Conversational and voice interface (the Time Agent)

The PS asks for an "LLM-based conversational or voice interface." Both halves must be visible.

Component	Implementation	Effort
Voice capture	Browser Web Speech API (<code>SpeechRecognition</code>). No ASR infrastructure, no model download, no API cost. Runs on the demo laptop's Chrome.	Small (2–3 h)
Transcript handling	The transcript enters the pipeline as a <code>field_reports</code> row with <code>source_type='agent'</code> . It takes exactly the same L3 → L4 → L5 path as an uploaded document.	None (reuse)
Clarification turns	When the matcher returns <code>REVIEW_REQUIRED</code> , the agent asks one disambiguating question ("Rack R03 or R07?") instead of dumping a queue item on the planner. The answer becomes additional context, and the event is re-scored.	Medium (half day)
LLM role	Optional, and only for ambiguous phrasing and clarification wording. Structured output with schema validation, and a deterministic fallback path under <code>DEMO_MODE</code> that makes zero API calls.	Small

The defensible framing: the Time Agent is not a chatbot bolted onto the side. It is a <i>second input surface onto the same pipeline</i>. Say this explicitly in the demo, and show a voice update and a document update producing the same event structure, the same evidence panel and the same audit row. That single comparison kills the "generic chatbot" objection outright. Rehearsal warning: the Web Speech API depends on network and microphone permissions. Pre-record a fallback clip and keep a typed transcript path ready. Never let a live microphone be a single point of failure in a judged demo.
--

9. API contract

POST	/api/v1/projects/{id}/schedule/import	multipart CSV/XLSX -> catalogue + DAG
GET	/api/v1/projects/{id}/activities	?discipline=&level=&status=
GET	/api/v1/activities/{id}	detail + planned/actual + float + is_critical
POST	/api/v1/reports	{source_type, content file, submitted_by}
		-> report_id, event_ids, latency_ms
GET	/api/v1/events/{id}	extracted fields + source spans
GET	/api/v1/events/{id}/candidates	ranked candidates + all four sub-scores + margin + evidence_json
GET	/api/v1/review-queue	?discipline= REVIEW REQUIRED + UNMATCHED
POST	/api/v1/events/{id}/confirm	{activity_id} planner approves/reassigns
POST	/api/v1/events/{id}/reject	{reason}
POST	/api/v1/projects/{id}/recompute	-> snapshot_id, finish_delta_days, critical_path_changes[], at_risk[]
GET	/api/v1/projects/{id}/critical-path	current critical chain + float distribution
GET	/api/v1/projects/{id}/snapshots/{sid}/diff	
GET	/api/v1/analytics/discipline-productivity	planned vs actual duration by discipline
GET	/api/v1/analytics/delay-causes	?discipline= ranked recurring causes
GET	/api/v1/analytics/variance	planned vs actual, by WBS node
GET	/api/v1/memory/similar	?description= historical actual durations
GET	/api/v1/audit	?activity_id=&from=&to=
GET	/api/v1/terminology-map	current version, entries, provenance
POST	/api/v1/agent/message	{text transcript, session_id}
		-> reply, event_id?, clarification?

The `latency_ms` field on the report response is deliberate. It is what the dashboard renders as the "report submitted → schedule updated" stopwatch, and it is how we demonstrate the PS's "near real time" clause with a measurement rather than an adjective.

10. Frontend — five screens, no more

Screen	Must contain	Owner
1. Ingest	Schedule upload; field report entry with four source tabs (paste text, upload sheet, Time Agent with mic button, diary stub). Live latency stopwatch on submit.	Frontend
2. Review queue	Ranked candidates per event. Per-signal evidence panel : the four sub-scores shown individually as bars, the margin to the runner-up called out explicitly, the highlighted source span in the original report, and the terminology-map entry that fired. Approve / reassign / reject.	Frontend
3. Schedule view	Gantt (frappe-gantt or dhtmlx-gantt — do not build a renderer), critical path highlighted, actuals overlaid on baseline, at-risk activities flagged after recompute.	Frontend
4. Analytics	Discipline-wise productivity, discipline-wise delay patterns, recurring delay causes ranked, variance by WBS, risk heatmap across the seven disciplines.	Frontend + AI
5. Audit & memory	Per-activity change history with old/new/source/score/actor; institutional-memory query returning historical actual durations for similar activities.	Backend + Frontend

Screen 2 is the screen the submission lives or dies on. It is the only place where "explainable" stops being a word and becomes something an evaluator can look at. If the week compresses, polish Screen 2 and let Screens 3–5 stay plain.

11. Synthetic data and the benchmark protocol

The PS permits synthetic data. It does not permit unmeasured claims — and "measured, not guessed" is already in the team's own definition of done. Making that literally true is a half-day of work and is the difference between evidence and assertion.

11.1 Generating the corpus

1. Author a baseline schedule of ~250–400 L5/L6 activities across all seven disciplines, with realistic OIL-style codes, WBS paths, locations and dependencies.
2. Generate field reports *from* known activities, so ground truth is known by construction.
3. **Corrupt deliberately** — this is the step that decides whether the benchmark means anything:
 - synonym substitution ("erect" → "install", "spool" for a line segment);
 - abbreviation and typo injection, OCR-style character confusions;
 - granularity mismatch (one plan activity reported as three partial field events);
 - **near-duplicate traps** — the same activity description at two different racks, which is the case the margin rule exists to catch;
 - reports for activities not in the plan at all, which must land in UNMATCHED;
 - discipline-ambiguous phrasing.
4. Hold out ~15% untouched. Nobody looks at it, tunes against it, or debugs with it until the final run.

11.2 What to report

Metric	Definition	Why it matters
Auto-match precision	Of events auto-linked without human review, the fraction linked to the correct activity.	The headline number. This is the false-positive risk an industrial user cares about.
Silent error count	Absolute count of wrong auto-links on the held-out set.	An absolute count is more honest and more persuasive than a percentage at this data scale.
Auto-match rate	Fraction of events resolved without a planner.	The efficiency claim. Meaningless without precision beside it.
Review rate / unmatched rate	Fractions routed to a human / preserved unlinked.	Shows the system degrades safely rather than guessing.
Recall on near-duplicate traps	How often the correct activity appears in the top 5 candidates.	Separates "retrieval failed" from "ranking failed" — a question a sharp evaluator will ask.
Margin distribution	Histogram of top-vs-second separation, correct against incorrect.	The visual that proves the margin rule works. One chart, high impact.
Median end-to-end latency	Report submitted → schedule updated, in seconds.	Evidences the "near real time" clause numerically.

Standing rule for the whole team, applied to the repository, the PPT and the video: no number is stated as a measured result unless it came out of an actual run in this repository. Thresholds and weights are documented as design choices, not as tuned accuracy. A single fabricated statistic that an evaluator probes and finds hollow costs more than every feature it was meant to support.

12. Development plan — 16 to 25 September

Four owners as briefed: **AI** (NLP, matching, benchmark), **BE** (data, API, audit, CPM), **FE** (all five screens, Time Agent UX), **QA** (synthetic data, tests, demo narrative). On a six-member team, the two additional members pair onto AI and FE, which are the critical path.

Date	Goal	Concrete deliverables	Owner
Tue 16	Lock and freeze	Scope table (§3) agreed and signed off. Data model (§5) frozen. Terminology map v0. Repo skeleton, environments, branch rules. Baseline schedule shape drafted.	All
Wed 17	Schedule side	CSV/XLSX ingestion → activity catalogue with discipline enum, WBS paths and dependency edges. DB migrations. Activity embeddings pre-computed and cached. <i>Parallel</i> : synthetic schedule of 250–400 activities complete.	BE / QA
Thu 18	Field side	Deterministic event extraction with source spans; normalisation pipeline consuming the terminology map; report ingestion for text and spreadsheet. <i>Parallel</i> : report generator with the six corruption modes, held-out split sealed.	AI / QA
Fri 19	The matcher	Blocking/k-NN retrieval; all four sub-scores; fixed-weight fusion; temporal multiplier; ranking; margin computation ; decision rule; all candidates and sub-scores persisted. Candidates API live.	AI / BE
Sat 20	Human in the loop	Review queue screen; per-signal evidence panel with margin called out and source span highlighted; confirm / reassign / reject; schedule actuals written; audit rows created on every write.	FE / BE
Sun 21	Close the loop	DAG build; CPM forward/backward pass; float; snapshot-and-diff on confirmed updates; at-risk downstream flagging; Gantt with critical path highlighted. End-to-end flow runs for the first time.	BE / FE
Mon 22	PS coverage	Time Agent: text + Web Speech voice + one clarification turn, through the same pipeline. Analytics: discipline-wise productivity and delay patterns, recurring delay-cause detection, variance. Diary stub if time permits.	FE / AI
Tue 23	Evidence day	Full benchmark run on the sealed held-out split. Calibrate the two thresholds, then freeze them. Produce the margin-distribution chart. pytest suite green. Numbers locked — no further tuning after today.	AI / QA
Wed 24	Narrative	Latency stopwatch wired into the UI. Demo script rehearsed end to end three times, including the failure-path rehearsal. Architecture diagram drawn with the reusable core separated from the OIL adapter. PPT and video.	All
Thu 25	Buffer + submit	Nothing new is built today. Fix only what broke. Submit with time in hand.	All

The one hard rule: the end-to-end flow must run on Sunday 21st, even if every piece inside it is ugly. A team with a working ugly pipeline on day six finishes; a team with four beautiful disconnected components on day eight does not.

12.1 Cut order if the week compresses

Stop wherever the clock forces you to. Items 1–6 constitute a submittable project; everything below 6 is polish.

1. Margin-based decision rule (defines what the matcher is)
2. Fixed fusion weights and the four sub-scores
3. Per-signal evidence panel (the explainability proof)
4. CPM recompute and critical-path diff (the differentiator)
5. Blocking / k-NN retrieval (the scale story)
6. Benchmark with held-out split (the evidence)
7. Terminology map surfaced in the UI
8. Latency stopwatch
9. Recurring delay-cause analytics
10. Voice input (keep — it closes a literal PS clause and costs hours)
11. Reusable-core framing in the architecture diagram (zero build cost, do it regardless)
12. Dual-threshold flagging
13. Scanned-diary stub (pure demo theatre; last)

13. Demo script — five minutes

Time	Action	The sentence to say
0:00	Show the baseline Gantt, critical path highlighted.	"This is the plan. It is precise, it is structured, and within a week of execution starting it is already wrong — because nothing on site reports progress in this vocabulary."
0:40	Paste a messy free-text daily report. Stopwatch starts.	"This is a real-shaped supervisor report. No activity code anywhere in it."
1:10	Show extraction with highlighted source spans, then the ranked candidates with the four sub-scores.	"Four independent signals, each shown separately. This is not a black-box number — you can see what produced it."
1:40	Auto-match case. High score, wide margin, auto-linked. Stopwatch stops — show the seconds.	"Submitted to updated in under N seconds. That is the near-real-time loop."
2:10	Ambiguous case. Same line, two racks. High top score, tiny margin. Held for review.	"The top score here is high — a threshold system would have accepted it and been wrong. We auto-match on <i>separation</i> , not on score. A close second place is the definition of ambiguity, so this goes to a human."
2:50	Planner confirms the correct one. Audit row appears.	"Old value, new value, source span, score, who decided, when. Every update, no exceptions."
3:10	CPM recompute fires. Critical path shifts; downstream at-risk activities light up; the projected finish date moves.	"This is the part that matters. That activity had zero float, so the critical path has moved, three downstream activities are now at risk, and mechanical completion slipped two days — surfaced now, not at the next monthly review."
3:45	Voice case. Speak an update into the Time Agent; it asks one clarifying question; same pipeline, same evidence panel, same audit row.	"Same pipeline. The conversational interface is a second input surface, not a separate chatbot."
4:15	Unmatched case. A report with no plan counterpart, preserved and visible.	"No match found — so we keep it and flag it. We never silently drop field data."
4:35	Analytics: discipline-wise delay patterns and recurring causes; the benchmark slide with the margin distribution.	"Across the held-out test set: this many auto-matched, this precision, this many silent errors. Measured in the repo, not estimated."

Rehearse the failure path too. Decide in advance what you do if the microphone fails, the LLM call times out, or the upload hangs. `DEMO_MODE` with deterministic fallbacks and a local dataset should mean the entire demo runs with the network unplugged. Test that literally — unplug it and run the full script once.

14. Risk register

Risk	Why it matters	Mitigation
Confident wrong link	Corrupts the progress record and every forecast built on it. The failure an industrial user fears most.	Margin rule; discipline penalty; temporal multiplier; review queue; per-signal evidence; silent-error count reported explicitly rather than buried.
Synthetic data too easy	A demo that looks better than the real problem is transparent to a domain evaluator and destroys credibility.	Six deliberate corruption modes including near-duplicate traps; sealed held-out split; show a hard case on stage, not only successes.
Indistinguishable from other submissions	Many teams are building the same consensus pipeline on the same stack.	CPM recompute; margin-based decisioning; measured benchmark; evidence UI. Lead with these three, in that order.
Domain misunderstanding	OIL evaluators run Primavera daily and will catch a wrong WBS or float claim instantly.	QA owner learns L1–L6 WBS, float, CPM and DPR conventions properly this week. Use correct terminology throughout. Never describe CPM as AI.
LLM dependency at demo time	Rate limits, timeouts and inconsistent output during a live judged run.	LLM is optional everywhere; schema validation on all structured output; DEMO_MODE deterministic path making zero API calls; full offline rehearsal.
Overbuilding the UI	The most common hackathon failure: time flows to screens instead of to the core.	Five screens, hard capped. Screen 2 gets the polish; everything else stays plain.
Integration left too late	Four good components that have never met do not become a demo in one night.	End-to-end flow mandatory on 21 September — six days out, with buffer.
Unbacked accuracy claims	One probe from an evaluator and the whole submission's credibility goes.	The standing rule in §11.2, applied to repo, PPT and video alike.

15. Judging criteria and evaluator Q&A

15.1 Mapping to the published criteria

Criterion	Where we score
Novelty	Margin-based decisioning under ambiguity; the closed loop from field text to recomputed critical path. Not "we used an LLM" — every team used an LLM.
Complexity	Multi-signal entity resolution with retrieve-then-rank, plus CPM on a dependency graph. Genuinely non-trivial and visibly so.
Clarity / detail	This document, the architecture diagram, and an evidence UI that makes the reasoning inspectable.
Feasibility	Entirely open-source and local: MiniLM, RapidFuzz, networkx, FastAPI, SQLite. Zero marginal cost, no paid tier, runs offline.
Practicability	Human-in-the-loop by design; audit trail; degrades safely to review rather than guessing. This is what makes it adoptable in an industrial setting.
Sustainability	No paid dependency; deterministic fallback for every AI component; institutional memory compounds in value as projects close.
Scale of impact	The domain-independent core versus the thin OIL adapter — drawn explicitly in the architecture diagram. The same engine serves any WBS-scheduled capital project.
User experience	Supervisor: speak an update, answer one question, done. Planner: a queue containing only genuinely ambiguous items, each with its reasoning attached.
Future work	The right-hand column of §3 — P6/PAIMANA integration, real OCR/ASR, mobile app, learned fusion once pilot labels exist, Monte Carlo risk.

15.2 Questions to have answers ready for

Question	Answer
"What happens when it matches the wrong activity?"	Lead with the margin rule, then the silent-error count from the held-out set, then the audit trail that makes any error traceable and reversible. Do not claim it never happens.
"Is this just semantic search with extra steps?"	No — semantic similarity is one of four signals and carries 0.25 weight. Structural context from the plan carries more, because it is more reliable than prose. And the decision is made on separation between candidates, not on similarity alone.
"How does it scale to a 40,000-activity schedule?"	Retrieve-then-rank: embeddings cached at ingestion, k-NN to ~20 candidates, full scoring on those only. Scoring cost is independent of schedule size.
"Where did your accuracy number come from?"	A named script, on a sealed held-out split, reproducible from the repo. Offer to run it live.
"Why not just fine-tune a model?"	No labelled corpus of real OIL field reports exists before a pilot. Fitting to synthetic data we authored ourselves would produce a number that means nothing. The architecture accepts a learned fusion layer the moment pilot labels exist — that is future work, stated as such.
"What does this give us that Primavera doesn't?"	Primavera holds the plan and accepts actuals. It has no mechanism to turn a supervisor's sentence into the right L5/L6 actual. That translation layer is the entire product.
"Does this replace our planners?"	No. It removes the unambiguous 60–70% so planners spend their time only on genuine ambiguity — and it shows them why each item is ambiguous.

16. Repository structure

progresssync/ backend/ app/ l1_ingestion/ l2_schedule/ l3_extraction/ l4_matching/ l5_decision/ l6_recompute/ l7_analytics/ l8_audit/ api/ agent/ data/ sample_schedule.csv sample_reports/ terminology_map.v1.yaml benchmark/ scripts/ generate_corpus.py run_benchmark.py demo_flow.py tests/ frontend/ src/components/ docs/ architecture.md decisions.md	schedule parsers, report intake, normalisation catalogue, DAG builder, CPM engine event extraction, entity + span capture blocking, four scorers, fusion, margin, ranking decision rule, review queue, planner actions snapshot, recompute, critical-path diff, at-risk variance, discipline stats, delay causes, memory append-only audit writer FastAPI routes, Pydantic schemas, app state Time Agent orchestration, DEMO_MODE fallback generated corpus + SEALED held-out split full pipeline, runnable end to end, offline pytest: CPM correctness, margin rule, unmatched preservation, audit completeness, API contract Ingest, ReviewQueue, EvidencePanel, GanttView, Analytics, AuditPanel, TimeAgent this document, maintained why each threshold and weight is what it is
---	---

17. Overall cost position

Every component is open-source and local. Embeddings run on a laptop CPU at this data scale via a MiniLM-class sentence-transformer; fuzzy matching is RapidFuzz; the graph and CPM are networkx; storage is SQLite for the demo with an identical SQLAlchemy schema against PostgreSQL for the production target; voice is the browser's own Web Speech API. The LLM is optional everywhere, confined to ambiguous-case phrasing and clarification wording, and has a deterministic zero-call fallback. **Total marginal cost of the feature set: nil.** The only real cost is engineering hours, which is why the cut order in \$12.1 matters far more than any budget line.

18. What "done" looks like on 24 September

- A reviewer watches one messy field sentence become a trustworthy schedule update in under two minutes, and can see exactly why the link was made.
- A high-confidence, wide-margin case auto-links and writes an actual.
- A high-score, narrow-margin case is correctly held for review — and the team can explain, without hedging, why that is the more important of the two.
- An unmatched observation stays visible and traceable.

- A confirmed update visibly moves the critical path and the projected finish date.
- A spoken update travels the same pipeline as an uploaded document and produces an identical evidence panel.
- Every update carries a complete audit row.
- Every stated number came out of a run in the repository.
- The whole demo runs with the network unplugged.

Sources and evidence base. Problem statement scope, organisation, disciplines and prototype guidance are from the SIH 2026 SIH26122 listing and Oil India Limited's published problem set. Section 2's landscape assessment is drawn from publicly visible SIH26122 implementations on GitHub as of September 2026, including repositories from competing teams; it reflects what those repositories state about their own scope and is a snapshot, not a complete census. Judging criteria are as published in SIH's official guidelines. All architecture, scoring weights, thresholds, schemas, timelines and strategic recommendations in this document are the team's own design decisions and should be presented as such, not as external findings.